

Lab Report: Advanced AI Architectures and Paradigms

Task 1: Fine-Tuning vs. Retrieval-Augmented Generation (RAG)

1. Core Concepts

- **Fine-Tuning:** The process of taking a pre-trained Large Language Model (LLM) and further training it on a specific, domain-targeted dataset to adjust its internal weights and parameters. This alters the model's behavior, tone, style, and domain vocabulary.
- **Retrieval-Augmented Generation (RAG):** An architectural pattern that enhances LLMs by connecting them to external knowledge sources. Before generating a response, the system retrieves relevant document chunks from an external database and injects them into the model's prompt as context.

2. Comprehensive Comparison

Dimension	Fine-Tuning	Retrieval-Augmented Generation (RAG)
Primary Purpose	Adapting style, tone, format, or specialized task learning.	Accessing dynamic, vast, or proprietary static knowledge.
Knowledge Updating	Static; requires costly retraining to update knowledge.	Dynamic; instant update by adding/editing external documents.
Hallucination Risk	Moderate to High (relies entirely on parametric)	Lower (anchored directly to retrieved source)

Dimension	Fine-Tuning	Retrieval-Augmented Generation (RAG)
	memory).	documents).
Traceability & Citations	Opaque; impossible to trace exact parameter origins.	Transparent; directly cites retrieved source passages.
Training & Maintenance Cost	High (requires GPUs, curated datasets, machine learning expertise).	Moderate (requires vector database and embedding pipeline setup).
Latency	Faster inference time (no external context retrieval step).	Slightly higher latency due to vector search and prompt overhead.

3. Real-World Suitability & Use Cases

- **When Fine-Tuning is Most Suitable:**
 - **Medical Coding & Legal Formatting:** When a model must consistently output responses following strict legal structures, niche nomenclature, or medical taxonomies (e.g., ICD-10 formatting).
 - **Specialized Tone / Persona:** Training a customer service bot to adopt a very specific brand voice or mimic a particular literary style.
- **When RAG is Most Suitable:**
 - **Enterprise Knowledge Portals:** An internal Q&A bot querying thousands of constantly updated company policy documents, Notion pages, or Google Drive files.
 - **Customer Support Databases:** E-commerce bots requiring real-time product inventory levels, order tracking info, and updated return guidelines.

Task 2: Workflows vs. AI Agents

1. Core Characteristics

- **AI Workflows:** Structured, deterministic sequences of execution paths where tasks are executed in a predefined order. Large Language Models are treated as individual function components within a fixed conditional pipeline (e.g., If-Else branching, explicit loops).
- **AI Agents:** Autonomous software entities driven by LLMs acting as the central reasoning engine. Agents evaluate objectives, dynamically plan sub-tasks, select appropriate tools (e.g., web search, APIs, code execution), evaluate intermediate outputs, and adapt execution paths self-correctively.

2. Strengths, Limitations, and Use Cases

Feature	AI Workflows	AI Agents
Control & Predictability	High; strictly follows defined logic paths.	Variable; non-deterministic decision-making.
Adaptability	Low; fails or halts when encountering unhandled edge cases.	High; dynamically changes strategy based on intermediate feedback.
Strengths	Consistent output, low latency, reliable, easy to debug.	Handles complex, open-ended multi-step problems autonomously.
Limitations	Rigid; requires explicit engineering for every edge case.	Potential infinite loops, higher cost, unpredictable behavior.
Common Use Cases	Document processing, data extraction, automated email triage.	Autonomous research, multi-step debugging, personal assistant tasks.

3. Real-World Applications

- **AI Workflow Example: Loan Application Screener**
An automated pipeline that receives an application PDF, parses text using OCR, runs an

LLM to extract key income fields, checks a database via a fixed SQL script, and triggers an email notification based on defined credit score thresholds.

- **AI Agent Example: Autonomous Market Research Assistant**

An agent given the goal "Analyze recent competitor pricing strategies." The agent formulates sub-queries, executes live web searches, scrapes target websites, identifies gaps, re-queries for missing details, synthesizes a summary report, and posts it to a Slack channel.

Task 3: RAG Research

1. Widely Used Chunking Techniques

Chunking is the process of breaking down large documents into smaller text segments before generating vector embeddings.

- **Fixed-Size Chunking:** Splits text into predefined character or token counts (e.g., 500 tokens) with a sliding overlap window (e.g., 50 tokens).
When & Why used: Simple, computationally efficient baseline; best for structured documents where structural context loss is minimal.
- **Recursive Character Chunking:** Recursively splits text using a hierarchy of separators (e.g., paragraph breaks `\n\n`, line breaks `\n`, spaces) until target chunk size is met.
When & Why used: The most popular general-purpose strategy; keeps semantically related sentences and paragraphs together.
- **Document-Based / Structural Chunking:** Parses native document structures (HTML headers `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, Markdown sections, JSON keys, or PDF section titles).

When & Why used: Essential for code repos, technical manuals, or API docs where structure dictates semantic meaning.

- **Semantic Chunking:** Computes embeddings at the sentence level and groups consecutive sentences together based on semantic similarity thresholds.
When & Why used: Used when context preservation is critical, preventing arbitrary splits in complex narratives or legal contracts.

2. Popular Vector Databases

Vector Database	Key Features & Strengths	When & Why Commonly Used
Pinecone	Fully managed, cloud-native serverless vector database with high scalability.	Used in enterprise applications requiring zero maintenance overhead and instant scaling.
ChromaDB	Lightweight, open-source, easily embeddable in Python/JS environments locally.	Standard choice for prototyping, local development, and small-to-medium datasets.
Qdrant	Written in Rust, high performance, robust payload filtering capabilities.	Selected for high-speed search requirements with heavy structured metadata filtering.
Milvus / Zilliz	Open-source distributed vector database designed for massive scale.	Ideal for enterprise-grade applications managing billions of vector embeddings.
pgvector (PostgreSQL)	Open-source extension adding vector similarity search to PostgreSQL.	Used when applications already use Postgres and want to avoid adding a new database engine.